Code smell

In computer programming, a **code smell** is any characteristic of source code that hints at a deeper problem.[1][2] Determining what a code smell is and is not is subjective, and varies by language, developer, and development methodology.

The term was popularized by Kent Beck on WardsWiki in the late 1990s.[3] Usage of the term increased after it was featured in the 1999 book *Refactoring: Improving the Design of Existing Code* by Martin Fowler.[4] It is also a term used by agile programmers.[5]

Definition

One way to look at smells is with respect to principles and quality: "Smells are certain structures in the code that indicate violation of fundamental design principles and negatively impact design quality".[6] Code smells are usually not bugs; they are not technically incorrect and do not prevent the program from functioning. Instead, they indicate weaknesses in design that may slow down development or increase the risk of bugs or failures in the future. Bad code smells can be an indicator of factors that contribute to technical debt.[1] Robert C. Martin calls a list of code smells a "value system" for software craftsmanship.[7]

Contrary to these severe interpretations, Cunningham's original definition was that a smell is a suggestion that something may be wrong, not evidence that there is already a problem.[3]

Often the deeper problem hinted at by a code smell can be uncovered when the code is subjected to a short feedback cycle, where it is refactored in small, controlled steps, and the resulting design is examined to see if there are any further code smells that in turn indicate the need for more refactoring. From the point of view of a programmer charged with performing refactoring, code smells are heuristics to indicate when to refactor, and what specific refactoring techniques to use. Thus, a code smell is a driver for refactoring.

Factors such as the understandability of code, how easy it is to be modified, the ease in which it can be enhanced to support functional changes, the code's ability to be reused in different settings, how testable the code is, and code reliability are factors that can be used to identify code smells.[8]

A 2015 study[1] utilizing automated analysis for half a million source code commits and the manual examination of 9,164 commits determined to exhibit "code smells" found that:

- There exists empirical evidence for the consequences of "technical debt", but there exists only anecdotal evidence as to *how*, *when*, or *why* this occurs.
- Common wisdom suggests that urgent maintenance activities and pressure to deliver features while prioritizing time-to-market over code quality are often the causes of such smells.

Tools such as Checkstyle, PMD, FindBugs, and SonarQube can automatically identify code smells.

Examples

Application-level smells

Duplicated code
: identical or very similar code that exists in more than one location.

Shotgun surgery
: a single change that needs to be applied to multiple classes at the same time.

Class-level smells

Large class, a.k.a. god object
: a class that contains too many types or contains many unrelated methods.

Refused bequest
: a class that overrides a method of a base class in such a way that the contract of the base class is not honored by the derived class, violating the Liskov substitution principle.

Excessive use of literals or magic numbers
: these should be coded as named constants, to improve readability and to avoid programming errors. Additionally, literals can and should be externalized into resource files/scripts, or other data stores such as databases where possible, to facilitate localization of software if it is intended to be deployed in different regions.[9]

Downcasting
: a type cast which breaks the abstraction model; the abstraction may have to be refactored or eliminated.[10]

Data clump
: Occurs when a group of variables are passed around together in various parts of the program. In general, this suggests that it would be more appropriate to formally group the different variables together into a single object, and pass around only the new object instead.[11][12]

Method-level smells

Too many parameters
: a long list of parameters is hard to read, and makes calling and testing the function complicated. It may indicate that the purpose of the function is ill-conceived and that the code should be refactored so responsibility is assigned in a more clean-cut way.[13]

See also

- Anti-pattern – Solution to a problem that may be commonly used but is generally a bad choice
- Design smell – Term in computer programming

- List of tools for static code analysis
- Software rot – Degradation or loss of the use of software over time

References

1. Tufano, Michele; Palomba, Fabio; Bavota, Gabriele; Oliveto, Rocco; Di Penta, Massimiliano; De Lucia, Andrea; Poshyvanyk, Denys (2015). "When and Why Your Code Starts to Smell Bad" (http://www.cs.wm.edu/~denys/pubs/ICSE'15-BadSmells-CRC.pdf) (PDF). *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*. pp. 403–414. CiteSeerX 10.1.1.709.6783 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.709.6783). doi:10.1109/ICSE.2015.59 (https://doi.org/10.1109%2FICSE.2015.59). ISBN 978-1-4799-1934-5. S2CID 59100195 (https://api.semanticscholar.org/CorpusID:59100195).
2. Fowler, Martin. "CodeSmell" (http://martinfowler.com/bliki/CodeSmell.html). *martinfowler.com/*. Retrieved 19 November 2014.
3. Beck, Kent. "Code Smells" (https://wiki.c2.com/?CodeSmell). *WikiWikiWeb*. Ward Cunningham. Retrieved 8 April 2020.
4. Fowler, Martin (1999). *Refactoring. Improving the Design of Existing Code* (https://archive.org/details/isbn_9780201485677). Addison-Wesley. ISBN 978-0-201-48567-7.
5. Binstock, Andrew (2011-06-27). "In Praise Of Small Code" (http://www.informationweek.com/architecture/in-praise-of-small-code/d/d-id/1098422). Information Week. Retrieved 2011-06-27.
6. Suryanarayana, Girish (November 2014). *Refactoring for Software Design Smells*. Morgan Kaufmann. p. 258. ISBN 978-0128013977.
7. Martin, Robert C. (2009). "17: Smells and Heuristics". *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall. ISBN 978-0-13-235088-4.
8. Suryanarayana, Girish, Ganesh Samarthyam, and Tushar Sharma. Refactoring for Software Design Smells : Managing Technical Debt / Girish Suryanarayana, Ganesh Samarthyam, Tushar Sharma. 1st edition. Waltham, Massachusetts ; Morgan Kaufmann, 2015. Print.
9. "Constants and Magic Numbers" (https://maximilianocontieri.com/code-smell-02-constants-and-magic-numbers). Retrieved 2020-11-03.
10. Miller, Jeremy. "Downcasting is a code smell" (https://web.archive.org/web/20190216193452/http://codebetter.com/jeremymiller/2006/12/26/downcasting-is-a-code-smell/). Archived from the original (http://codebetter.com/jeremymiller/2006/12/26/downcasting-is-a-code-smell/) on 16 February 2019. Retrieved 4 December 2014.
11. Fowler, Martin. "DataClump" (https://martinfowler.com/bliki/DataClump.html). Retrieved 2017-02-03.
12. "Design Patterns and Refactoring" (https://sourcemaking.com/refactoring/smells/data-clumps). *sourcemaking.com*. Retrieved 2017-02-04.
13. "Code Smell 10 - Too Many Arguments" (https://mcsee.hashnode.dev/code-smell-10-too-many-arguments).

Further reading

- Garousi, Vahid; Küçük, Barış (2018). "Smells in software test code: A survey of knowledge in industry and academia". *Journal of Systems and Software*. **138**: 52–81. doi:10.1016/j.jss.2017.12.013 (https://doi.org/10.1016%2Fj.jss.2017.12.013).
- Sharma, Tushar; Spinellis, Diomidis (2018). "A survey on software smells" (https://zenodo.org/record/1997449). *Journal of Systems and Software*. **138**: 158–173. doi:10.1016/j.jss.2017.12.034 (https://doi.org/10.1016%2Fj.jss.2017.12.034).

External links

- "CodeSmell" (https://martinfowler.com/bliki/CodeSmell.html). *martinfowler.com*. Retrieved 2022-03-01.
- Boundy, David, Software cancer: the seven early warning signs (http://dl.acm.org/citation.cfm?id=156632) or here (https://www.academia.edu/2303865/Software_cancer_the_seven_early_warning_signs), ACM SIGSOFT Software Engineering Notes, Vol. 18 No. 2 (April 1993), Association for Computing Machinery, New York, NY, USA